

uni-app分类列表：Storage存储与预览页缓存读取完整实现

在uni-app开发中，将分类列表数据存入本地缓存（Storage），并在预览页面读取缓存展示，是提升页面加载速度、减少重复接口请求的常用方案。本文将详细讲解从数据存储、缓存管理到预览页读取展示的全流程，包含异常处理、数据更新等关键细节，确保方案稳定可靠。

一、核心原理与适用场景

1. 核心原理

利用uni-app提供的 `uni.setStorageSync` / `uni.getStorageSync` 等Storage相关API，将分类列表接口返回的数据以键值对形式存入本地缓存。预览页面加载时，优先从缓存中读取数据展示，同时可根据需求决定是否同步请求最新数据更新缓存，实现“缓存优先、同步更新”的效果。

2. 适用场景

- 分类数据更新频率低（如商品分类、资讯分类），无需实时从接口获取；
- 页面跳转频繁（如从分类页到预览页），重复请求接口会导致加载延迟；
- 弱网或离线场景下，需保障基础数据的正常展示。

二、完整实现流程

以“商品分类列表”为例，实现“分类页获取数据并存储→预览页读取缓存展示”的完整流程，包含数据存储、缓存读取、异常处理三个核心环节。

1. 步骤1：分类列表页（数据获取与Storage存储）

分类页负责从接口获取分类数据，存储到Storage中，同时支持数据更新时同步刷新缓存。

1.1 页面结构（pages/category/index.vue）

简单展示分类列表，点击分类项跳转到预览页，并携带分类ID用于精准读取数据。

Code block

```
1 <template>
2   <view class="category-page">
3     <view class="page-title">商品分类</view>
4
5
```

```

6      <view class="loading" v-if="loading">加载中...</view>
7
8
9      <view class="category-list" v-else>
10         <view class="category-item"
11             v-for="item in categoryList"
12             :key="item.id"
13             @click="goToPreview(item.id)">
14             <view class="category-name">{{ item.name }}</view>
15             <image class="arrow-icon" src="/static/icons/arrow-right.png"
mode="widthFix"></image>
16         </view>
17     </view>
18 </view>
19 </template>

```

1.2 核心逻辑（数据获取与存储）

重点实现 `getCategoryData` 方法，完成“接口请求→数据处理→Storage存储”的流程，同时添加缓存过期判断（可选）。

Code block

```

1  <script>
2  import { getCategoryList } from '@api/category.js'; // 分类接口封装
3
4  export default {
5      data() {
6          return {
7              categoryList: [], // 分类列表数据
8              loading: true, // 加载状态
9              storageKey: 'goods_category_list', // Storage存储键名（统一管理）
10             cacheExpire: 86400000 // 缓存过期时间（24小时，单位：毫秒，可选）
11         };
12     },
13     onLoad() {
14         // 页面加载时获取分类数据（优先读缓存，无缓存则请求接口）
15         this.getCategoryData();
16     },
17     methods: {
18         /**
19          * 获取分类数据（接口请求+Storage存储）
20          */
21         async getCategoryData() {
22             try {
23                 this.loading = true;
24

```

```

25      // 1. 先尝试读取缓存 (优化: 可判断缓存是否过期)
26      const cacheData = this.getCategoryFromStorage();
27      if (cacheData) {
28          this.categoryList = cacheData;
29          // 后台同步请求最新数据, 更新缓存 (不阻塞页面展示)
30          this.syncCategoryData();
31          return;
32      }
33
34      // 2. 无缓存时请求接口
35      const res = await getCategoryList();
36      if (res.code === 200) {
37          this.categoryList = res.data;
38          // 3. 将数据存入Storage (包含存储时间, 用于判断过期)
39          this.saveCategoryToStorage(res.data);
40      } else {
41          uni.showToast({ title: res.msg || '获取分类失败', icon: 'none' });
42      }
43  } catch (error) {
44      console.error('获取分类数据异常: ', error);
45      uni.showToast({ title: '网络异常', icon: 'none' });
46  } finally {
47      this.loading = false;
48  }
49  },
50
51  /**
52   * 从Storage读取分类数据 (含过期判断)
53   * @returns {Array|null} 分类数据 (过期返回null)
54   */
55  getCategoryFromStorage() {
56      try {
57          const storageData = uni.getStorageSync(this.storageKey);
58          if (!storageData) return null;
59
60          // 解析缓存数据 (存储时包含data和time)
61          const { data, time } = JSON.parse(storageData);
62          // 判断缓存是否过期 (当前时间 - 存储时间 > 过期时间则过期)
63          if (Date.now() - time > this.cacheExpire) {
64              this.clearCategoryStorage(); // 清除过期缓存
65              return null;
66          }
67          return data;
68      } catch (err) {
69          console.error('读取分类缓存失败: ', err);
70          this.clearCategoryStorage(); // 读取失败时清除缓存
71          return null;

```

```

72     }
73   },
74
75   /**
76    * 将分类数据存入Storage (格式: {data: 数据, time: 存储时间})
77    * @param {Array} data - 分类列表数据
78    */
79   saveCategoryToStorage(data) {
80     try {
81       const storageData = JSON.stringify({
82         data: data,
83         time: Date.now() // 记录存储时间
84       });
85       uni.setStorageSync(this.storageKey, storageData);
86     } catch (err) {
87       console.error('存储分类缓存失败: ', err);
88       uni.showToast({ title: '缓存存储失败', icon: 'none' });

```

1.3 样式补充

Code block

```

1  <style scoped>
2  .category-page {
3    padding: 20rpx;
4    background-color: #fafafa;
5  }
6  .page-title {
7    font-size: 36rpx;
8    font-weight: bold;
9    margin-bottom: 30rpx;
10   color: #333;
11 }
12 .loading {
13   text-align: center;
14   padding: 50rpx 0;
15   color: #999;
16   font-size: 24rpx;
17 }
18 .category-list {
19   width: 100%;
20 }
21 .category-item {
22   display: flex;
23   justify-content: space-between;
24   align-items: center;
25   padding: 24rpx;

```

```

26     background-color: #fff;
27     border-radius: 16rpx;
28     margin-bottom: 20rpx;
29     box-shadow: 0 2rpx 8rpx rgba(0,0,0,0.03);
30 }
31 .category-name {
32     font-size: 32rpx;
33     color: #333;
34 }
35 .arrow-icon {
36     width: 24rpx;
37     height: 24rpx;
38     opacity: 0.6;
39 }
40 </style>

```

2. 步骤2：分类预览页（读取Storage缓存并展示）

预览页通过分类ID从Storage缓存中筛选对应分类的详细数据（假设分类数据包含详情字段），实现快速展示。

2.1 页面结构（pages/category/preview.vue）

展示分类详情信息，若缓存读取失败则提示并提供重试入口。

Code block

```

1  <template>
2    <view class="preview-page">
3
4      <view class="loading" v-if="loading">加载中...</view>
5
6
7      <view class="category-detail" v-else-if="currentCategory">
8        <view class="detail-title">{{ currentCategory.name }}</view>
9        <view class="detail-desc">{{ currentCategory.desc }}</view>
10       <view class="detail-count">包含商品：{{ currentCategory.goodsCount }} 件
</view>
11     </view>
12
13
14     <view class="error-tips" v-else>
15       <text>分类数据获取失败</text>
16       <button class="retry-btn" @click="reloadData">重试</button>
17     </view>
18   </view>
19 </template>

```

2.2 核心逻辑（缓存读取与数据筛选）

在 `onLoad` 中接收分类ID，从Storage读取完整分类列表，筛选出当前分类数据；若缓存无数据，可触发接口请求并更新缓存。

Code block

```
1  <script>
2  import { getCategoryList } from '@api/category.js';
3
4  export default {
5    data() {
6      return {
7        loading: true, // 加载状态
8        currentCategory: null, // 当前分类详情数据
9        categoryId: '', // 接收的分类ID
10       storageKey: 'goods_category_list' // 与分类页统一的Storage键名
11     };
12   },
13   onLoad(options) {
14     // 1. 接收分类页传递的分类ID（注意转换为数字类型，与接口一致）
15     this.categoryId = Number(options.categoryId);
16     // 2. 读取缓存并筛选当前分类数据
17     this.getCategoryFromStorage();
18   },
19   methods: {
20     /**
21      * 从Storage读取分类数据并筛选当前分类
22      */
23     async getCategoryFromStorage() {
24       try {
25         this.loading = true;
26         // 1. 读取缓存中的完整分类列表
27         const storageData = uni.getStorageSync(this.storageKey);
28         if (!storageData) throw new Error('缓存中无分类数据');
29
30         // 2. 解析缓存数据
31         const { data } = JSON.parse(storageData);
32         // 3. 根据分类ID筛选当前分类数据
33         const targetCategory = data.find(item => item.id === this.categoryId);
34         if (targetCategory) {
35           this.currentCategory = targetCategory;
36         } else {
37           throw new Error('未找到对应分类');
38         }
39       } catch (err) {
```

```

40     console.error('读取分类缓存异常: ', err);
41     // 缓存读取失败时, 请求接口获取最新数据
42     await this.getCategoryFromApi();
43   } finally {
44     this.loading = false;
45   }
46 },
47
48 /**
49  * 从接口获取分类数据 (缓存读取失败时降级方案)
50  */
51 async getCategoryFromApi() {
52   try {
53     const res = await getCategoryList();
54     if (res.code === 200) {
55       // 1. 筛选当前分类数据
56       const targetCategory = res.data.find(item => item.id ===
this.categoryId);
57       if (targetCategory) {
58         this.currentCategory = targetCategory;
59         // 2. 更新缓存 (同步到Storage, 供后续使用)
60         this.saveCategoryToStorage(res.data);
61       } else {
62         uni.showToast({ title: '分类不存在', icon: 'none' });
63       }
64     } else {
65       uni.showToast({ title: res.msg || '获取分类失败', icon: 'none' });
66     }
67   } catch (err) {
68     console.error('接口请求分类数据失败: ', err);
69     uni.showToast({ title: '网络异常, 无法获取数据', icon: 'none' });
70   }
71 },
72
73 /**
74  * 保存分类数据到Storage (与分类页逻辑一致, 可封装为公共方法)
75  * @param {Array} data - 分类列表数据
76  */
77 saveCategoryToStorage(data) {
78   try {
79     const storageData = JSON.stringify({
80       data: data,
81       time: Date.now()
82     });
83     uni.setStorageSync(this.storageKey, storageData);
84   } catch (err) {
85     console.error('存储分类缓存失败: ', err);

```

```
86     }  
87     }.
```

2.3 样式补充

Code block

```
1  <style scoped>  
2  .preview-page {  
3    padding: 20rpx;  
4    background-color: #fafafa;  
5    min-height: 100vh;  
6  }  
7  .loading {  
8    text-align: center;  
9    padding: 100rpx 0;  
10   color: #999;  
11   font-size: 24rpx;  
12 }  
13 .category-detail {  
14   background-color: #fff;  
15   padding: 30rpx;  
16   border-radius: 16rpx;  
17   box-shadow: 0 2rpx 8rpx rgba(0,0,0,0.03);  
18 }  
19 .detail-title {  
20   font-size: 36rpx;  
21   font-weight: bold;  
22   color: #333;  
23   margin-bottom: 20rpx;  
24 }  
25 .detail-desc {  
26   font-size: 26rpx;  
27   color: #666;  
28   line-height: 40rpx;  
29   margin-bottom: 30rpx;  
30 }  
31 .detail-count {  
32   font-size: 24rpx;  
33   color: #999;  
34   padding-top: 20rpx;  
35   border-top: 1rpx solid #f5f5f5;  
36 }  
37 .error-tips {  
38   text-align: center;  
39   padding: 100rpx 0;  
40   color: #999;  
41   font-size: 24rpx;
```



```
42 }
43 .retry-btn {
44   margin-top: 30rpx;
45   background-color: #2c83ff;
46   color: #fff;
47   font-size: 24rpx;
48   padding: 12rpx 40rpx;
49   border-radius: 30rpx;
50 }
51 </style>
```

3. 步骤3：接口封装（api/category.js）

统一封装分类列表接口，降低页面逻辑耦合度。

Code block

```
1  // 基地址（可全局配置中定义）
2  const baseUrl = 'https://api.example.com/uni-app';
3
4  /**
5   * 获取商品分类列表接口
6   * @returns {Promise} 请求Promise
7   */
8  export const getCategoryList = () => {
9    return new Promise((resolve, reject) => {
10      uni.request({
11        url: `${baseUrl}/category/list`,
12        method: 'GET',
13        header: {
14          'Content-Type': 'application/json'
15        },
16        success: (res) => {
17          if (res.statusCode === 200) {
18            resolve(res.data);
19          } else {
20            reject(new Error(`请求失败：HTTP ${res.statusCode}`));
21          }
22        },
23        fail: (err) => {
24          reject(new Error(`网络异常：${err.errMsg}`));
25        }
26      });
27    });
28  };
```

三、关键注意事项

1. 数据类型一致性

Storage存储的是字符串类型，因此存入时需用 `JSON.stringify` 序列化，读取时用 `JSON.parse` 反序列化；同时，分类ID传递时需注意类型转换（如从字符串转为数字），避免因类型不匹配导致筛选失败。

2. 缓存过期与更新策略

- 建议为缓存添加过期时间，避免长期使用旧数据（如本文中24小时过期）；
- 分类数据更新时（如后台添加新分类），需在分类页主动调用 `saveCategoryToStorage` 方法更新缓存，或通过接口返回的“更新时间”判断是否需要刷新；
- 可在APP启动时或特定时机（如用户下拉刷新）触发缓存同步，确保数据时效性。

3. 异常处理与降级方案

必须处理以下异常场景，避免页面崩溃或展示异常：

1. Storage读取失败（如缓存损坏、存储空间不足）：降级为直接请求接口；
2. 缓存中无对应分类数据：提示用户并提供重试入口；
3. 接口请求与缓存均失败：展示友好的错误提示，引导用户检查网络。

4. 存储键名统一管理

将Storage键名（如 `goods_category_list`）统一定义在公共配置文件中（如 `config/storage.js`），避免不同页面键名不一致导致的读取失败，示例：

Code block

```
1  // config/storage.js
2  export const STORAGE_KEY = {
3    GOODS_CATEGORY: 'goods_category_list',
4    USER_INFO: 'user_info'
5  }
6
7  // 页面中引入使用
8  import { STORAGE_KEY } from '@config/storage.js';
9  this.storageKey = STORAGE_KEY.GOODS_CATEGORY;
```

5. 存储空间限制

不同平台的Storage存储空间存在限制（如微信小程序单个键值对不超过16KB，总存储空间不超过10MB），分类列表数据若过大（如包含大量商品信息），需拆分存储或仅存储核心字段，详情字段通

过接口按需获取。

6. 敏感数据不存储

Storage是本地明文存储，不适合存储敏感数据（如用户token、密码），分类列表等非敏感公共数据可安全存储。

四、扩展场景

1. 复杂分类数据存储

若分类数据包含层级结构（如一级分类下有二级分类），存储逻辑不变，预览页读取后可通过递归或循环筛选对应层级的分类数据。

2. 多端适配

uni-app的Storage API在各平台（小程序、APP、H5）均兼容，无需额外适配；H5端需注意浏览器隐私模式下Storage可能不可用，需添加异常捕获。

3. 缓存清理

可在APP的“设置”页面添加“清除缓存”功能，调用 `uni.removeStorageSync` 或 `uni.clearStorageSync` 清除分类缓存，示例：

Code block

```
1  // 清除分类缓存
2  clearCategoryCache() {
3    try {
4      uni.removeStorageSync(STORAGE_KEY.GOODS_CATEGORY);
5      uni.showToast({ title: '缓存已清除' });
6    } catch (err) {
7      console.error('清除缓存失败：', err);
8    }
9  }
```

五、总结

将分类列表存入Storage并在预览页读取展示，核心是利用本地缓存减少接口请求，提升页面加载速度。通过“缓存优先+接口降级”的策略，结合完善的异常处理和缓存管理机制，既能保障用户体验，又能确保数据的时效性和可靠性。实际开发中可根据分类数据的更新频率、大小等特点，灵活调整缓存过期时间和存储策略，实现最优效果。

（注：文档部分内容可能由 AI 生成）